FINGERPRINT BIOMETRIC ATTENDANCE SYSTEM

MASTER OF TECHNOLOGY IN
COMPUTER SCIENCE & ENGINEERING WITH SPECIALIZATION IN
DATA SCIENCE AND ARTIFICIAL INTELLIGENCE

Submitted by,

ANAGHA R S

ANNA SEBASTIAN

Under the guidance of

DR. BIJOY A JOSE

Department of Computer Science & Engineering

COCHIN UNIVERSITY OF SCIENCE AND TECHNOLOGY

NOVEMBER 2024

ABSTRACT

The Biometric Fingerprint Attendance System aims to streamline attendance monitoring in educational institutions and corporate environments through advanced biometric technology. By integrating a fingerprint sensor, the system ensures secure and accurate identification of individuals. Leveraging MQTT for efficient communication, the system captures fingerprint data upon interaction and securely stores the attendance information in a cloud-based database. Each attendance record is timestamped, offering a reliable log of attendance events. A web-based interface allows administrators to manage attendance records, view real-time data, and generate reports. The system's use of cloud storage ensures accessibility and scalability, making it a robust solution for managing attendance across large institutions. The Biometric Fingerprint Attendance System enhances security, reduces manual errors, and provides a reliable method for tracking attendance in real time.

TABLE OF CONTENTS

1. INTRODUCTION

The Biometric Fingerprint Attendance System is designed to simplify and enhance attendance tracking in educational institutions and corporate environments. Traditional methods of recording attendance, such as manual sign-ins or ID card swipes, can be prone to errors or misuse. This system utilizes fingerprint sensors to uniquely identify individuals, ensuring accurate and secure attendance logging. Upon fingerprint detection, the system records the time and stores the data in a cloud-based database for easy access and management. By integrating MQTT for efficient data communication, the system ensures real-time synchronization between the fingerprint sensor and the cloud, allowing administrators to view up-to-date attendance records through a web interface. Additionally, the system provides notifications and detailed reports, offering a reliable and streamlined solution for managing attendance.

1.1 Aim

The primary aim of the Biometric Fingerprint Attendance System project is to implement a secure and efficient attendance-tracking solution using fingerprint recognition technology. By integrating a fingerprint sensor for accurate identification and an MQTT-based communication system for real-time data transfer, the project ensures reliable logging of attendance data. With a cloud-based database for secure storage and a web interface for easy management, this system simplifies attendance monitoring and enhances accuracy, offering a robust solution for educational institutions and corporate environments.

1.2 Objective

1. Fingerprint Detection: Implement a system to accurately capture and identify individual fingerprints for attendance logging.

2. Real-Time Data Transfer: Utilize MQTT protocol to ensure efficient and real-time communication of fingerprint data between the sensor and the cloud database.

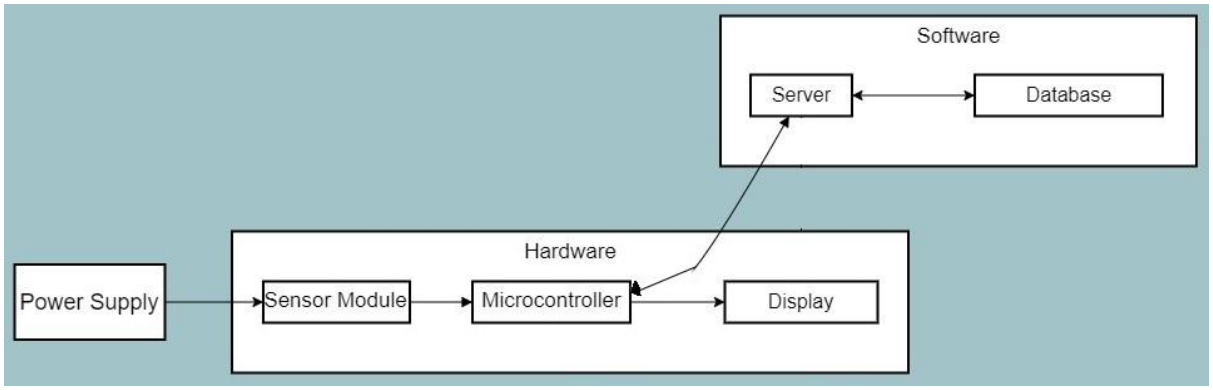
3. Timestamping: Ensure each attendance entry is accompanied by an exact date and time stamp for precise record-keeping.

4. Cloud Storage: Store attendance records securely in a cloud-based database, ensuring easy access, scalability, and data integrity.

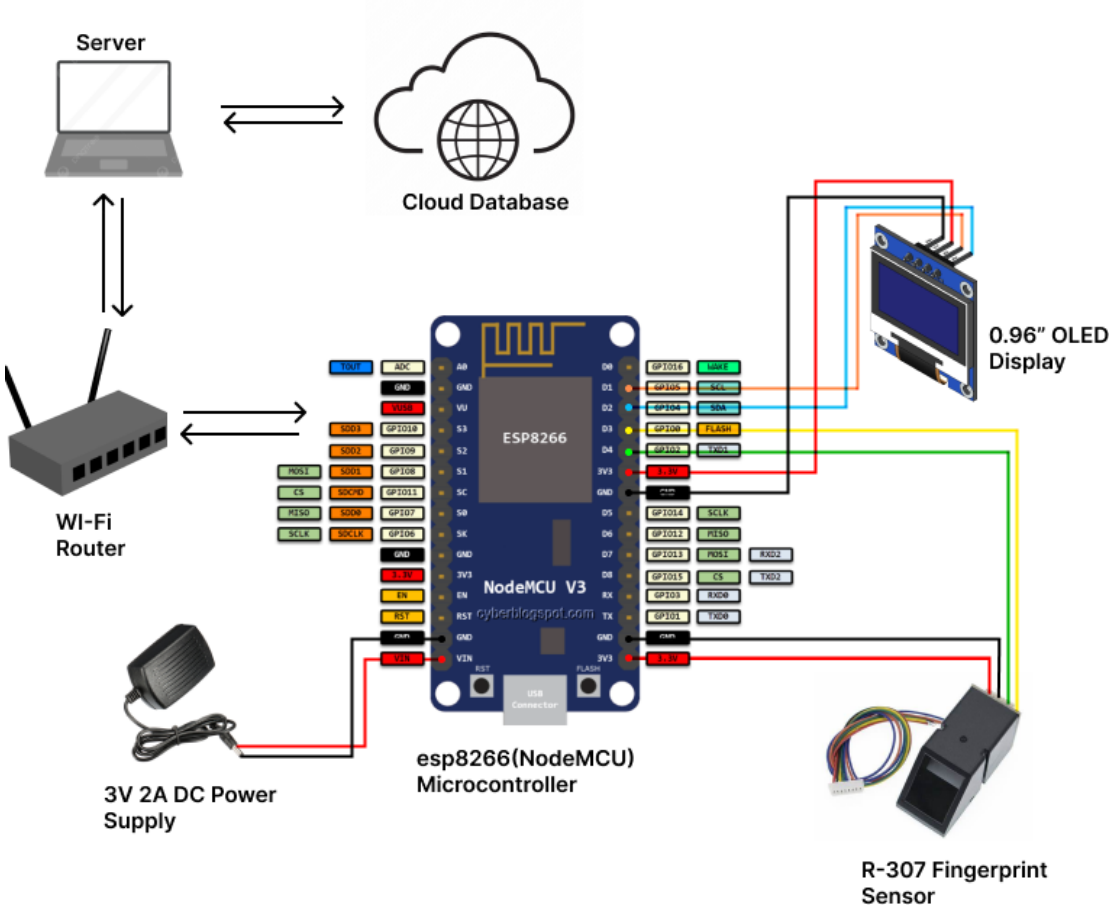
5. Web Interface: Develop a user-friendly web platform for administrators to manage attendance records, generate reports, and monitor real-time data.

2. DIAGRAMS

2.1 HIGH LEVEL DIAGRAM



2.2 LOW LEVEL DIAGRAM



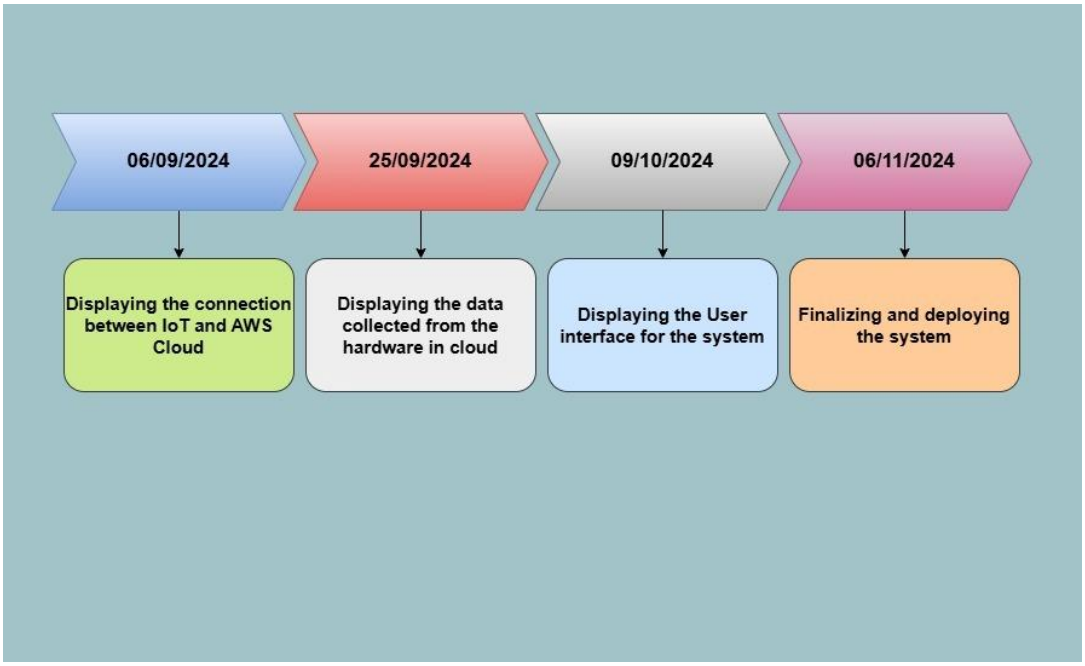
3. MATERIALS USED

- **NodeMCU(ESP8266)**: Serves as the main microcontroller, handling fingerprint data processing, managing sensor interactions, and facilitating MQTT communication for real-time data transfer.
- **R307 Fingerprint Sensor**: Captures fingerprint data for secure and accurate attendance logging, ensuring unique identification of individuals.
- **LED Display**: Provides visual feedback for users, indicating successful fingerprint scans or errors during attendance registration.
- **AWS Cloud Services**: Used for cloud storage, providing a secure and scalable database solution to store recorded fingerprint templates.
- **Zero PCB**: Acts as the hardware platform for assembling and connecting various electronic components, ensuring the system's structural stability.
- **MQTT Broker**: Facilitates efficient communication between the fingerprint sensor and cloud-based services, ensuring real-time data transmission.
- **Website**: A web-based interface that allows administrators to access, manage, and monitor attendance records, including functionalities like adding, deleting, and verifying users.

SN	**Component Name**	**Quantity**	**Price**
1	NodeMCU(ESP8266)	1	394/-
2	R 307 Fingerprint Sensor	1	1080/-
3	LED Display	1	249/-
4	Zero PCB	1	30/-
Total			1753/-

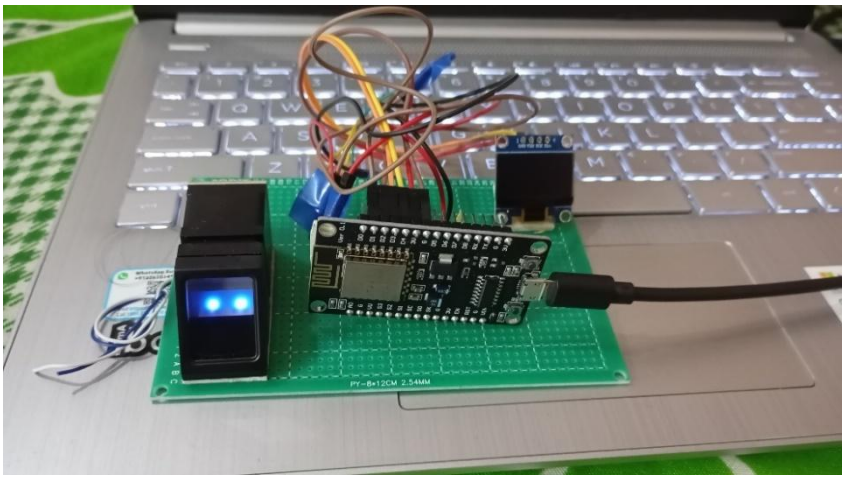
4. TIMELINE

- **Week 1 (Aug 27 - Sep 6):** Cloud Connectivity Setup & MQTT Demonstration: Establishing a communication pipeline using MQTT and validating successful data transmission to the cloud.
- **Week 2 (Sep 6 - Sep 25):** Displaying the data collected from the hardware fingerprint sensor on AWS cloud.
- **Week 3 (Sep 25 - Oct 9):** Developing and Displaying the web interface for viewing biometric attendance details.
- **Week 4 (Oct 9 - Oct 23):** Connecting cloud with attendance management system: Integrate the cloud platform with the web interface, allowing fingerprint data and attendance records from the NodeMCU to be accessible and visualized through the website.
- **Week 5(Oct 23 – Nov 6):** Finalizing and deploying the monitoring system: Complete final tasks, including testing. Implement MySQL connection, then fully deploy the system for use.



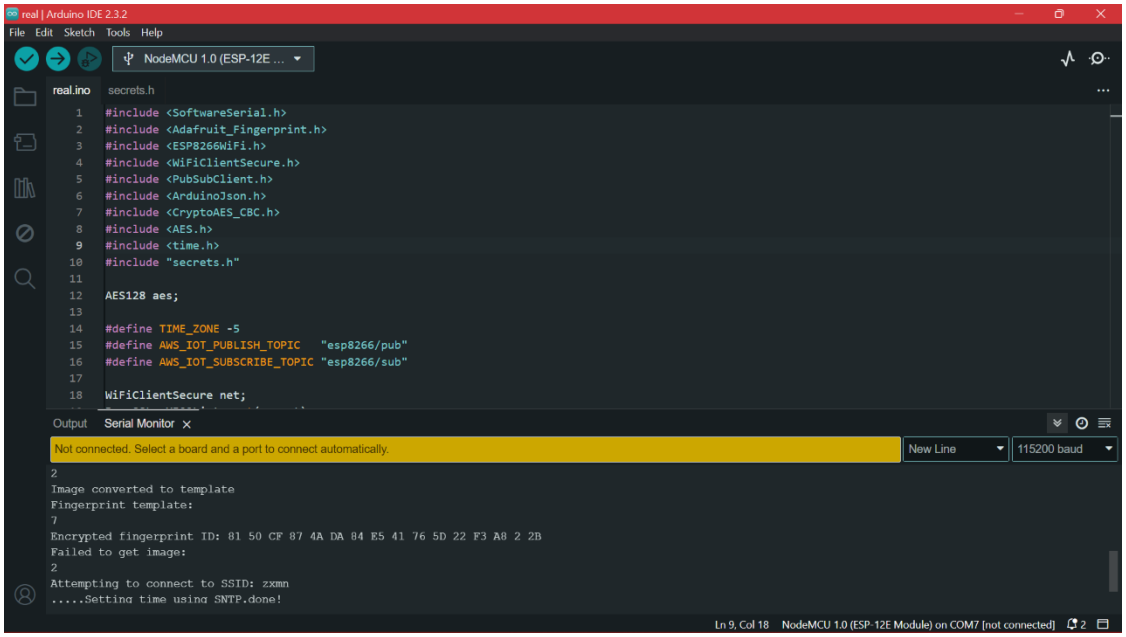
5. SCREENSHOTS

Hardware setup



Shows the high-level structure of biometric attendance system, illustrating how the NodeMCU(esp8266), R307 fingerprint sensor and AWS cloud interact through MQTT to detect fingerprint and store data securely.

Cloud setup Interface



This shows the Arduino code for sending fingerprint data to AWS.

Arduino Code :

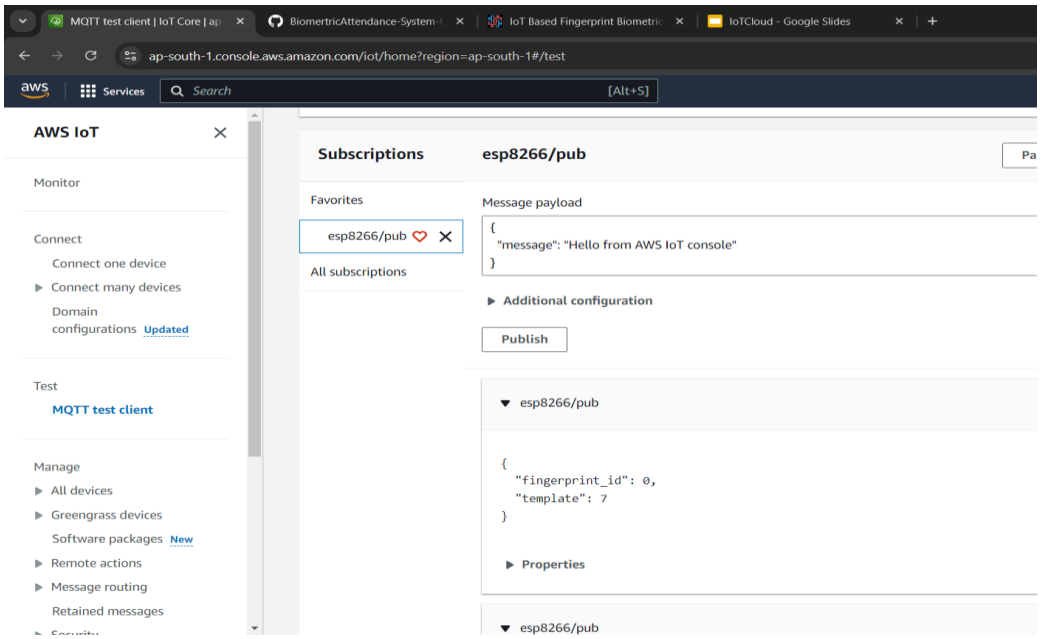
```
#include <SoftwareSerial.h>
#include <Adafruit_Fingerprint.h>
#include <ESP8266WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <time.h>
#include "secrets.h"

#define TIME_ZONE -5

#define AWS_IOT_PUBLISH_TOPIC   "user/fingerprint/response"
#define AWS_IOT_SUBSCRIBE_TOPIC "user/fingerprint/request"

WiFiClientSecure net;
BearSSL::X509List cert(cacert);
BearSSL::X509List client_crt(client_cert);
BearSSL::PrivateKey key(privkey);
PubSubClient client(net);

SoftwareSerial mySerial(D2, D1); // RX, TX
Adafruit_Fingerprint finger = Adafruit_Fingerprint(&mySerial);

int currentId = 1; // Starting ID for enrollment
const int maxId = 127; // Maximum ID limit for the fingerprint sensor

unsigned long lastMillis = 0; // Initialize lastMillis
char jsonBuffer[256]; // Buffer to hold the JSON string

time_t now;
time_t nowish = 1510592825;

void NTPConnect(void) {
  Serial.print("Setting time using SNTP");
  configTime(TIME_ZONE * 3600, 0 * 3600, "pool.ntp.org", "time.nist.gov");
  now = time(nullptr);
  while (now < nowish) {
    delay(500);
    Serial.print(".");
    now = time(nullptr);
  }
  Serial.println("done!");
  struct tm timeinfo;
  gmtime_r(&now, &timeinfo);
  Serial.print("Current time: ");
  Serial.print(asctime(&timeinfo));
}

void messageReceived(char *topic, byte *payload, unsigned int length) {
  Serial.print("Received [");
```

```
  Serial.print(topic);
  Serial.print("]: ");
  String receivedMessage;
  for (int i = 0; i < length; i++) {
    receivedMessage += (char)payload[i];
  }
  Serial.println(receivedMessage);

  // Parse the message and check for the "username" field
  StaticJsonDocument<512> doc;
  DeserializationError error = deserializeJson(doc, receivedMessage);

  if (error) {
    Serial.println("Failed to parse incoming message.");
    return;
  }

  // Check if the "username" field exists and is "enroll"
  const char* username = doc["username"];

  if (username && strcmp(username, "enroll") == 0) {
    // If the username is "enroll", call the enrollFingerprint function
    Serial.println("Received enrollment request.");
    enrollFingerprint();
  } else {
    Serial.println("Invalid or unrecognized username.");
  }
}

void connectAWS() {
  delay(3000);
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  Serial.println(String("Attempting to connect to SSID: ") + String(WIFI_SSID));

  while (WiFi.status() != WL_CONNECTED) {
    Serial.print(".");
    delay(1000);
  }

  Serial.println("Connected to WiFi");

  NTPConnect();

  net.setTrustAnchors(&cert);
  net.setClientRSACert(&client_crt, &key);

  client.setServer(MQTT_HOST, 8883);
  client.setCallback(messageReceived);

  Serial.println("Connecting to AWS IoT");
```

```
  while (!client.connect(THINGNAME)) {
    Serial.print(".");
    delay(1000);
  }

  if (client.connected()) {
    Serial.println("AWS IoT Connected!");
    client.subscribe(AWS_IOT_SUBSCRIBE_TOPIC); // Subscribe to the AWS IoT topic
  } else {
    Serial.println("AWS IoT Timeout or connection failed.");
  }
}

void publishFingerprintId(int fingerprint_id) {
  // Only send the fingerprint ID
  StaticJsonDocument<512> doc;
  doc["fingerprint_id"] = fingerprint_id;

  char jsonBuffer[512];
  serializeJson(doc, jsonBuffer);

  Serial.println("Publishing fingerprint ID to AWS IoT...");

  bool result = client.publish(AWS_IOT_PUBLISH_TOPIC, jsonBuffer);

  if (result) {
    Serial.println("Fingerprint ID successfully published:");
    Serial.println(jsonBuffer);
  } else {
    Serial.println("Failed to publish fingerprint ID to AWS.");
  }
}

void setup() {
  Serial.begin(115200);
  mySerial.begin(57600);
  Serial.println("Fingerprint System Ready");

  connectAWS();

  finger.begin(57600);
  if (finger.verifyPassword()) {
    Serial.println("Fingerprint sensor found!");
  } else {
    Serial.println("No fingerprint sensor found. Please check your connections.");
    while (1);
  }
}

void loop() {
  now = time(nullptr);

                                        // Keep MQTT client loop active to maintain
```

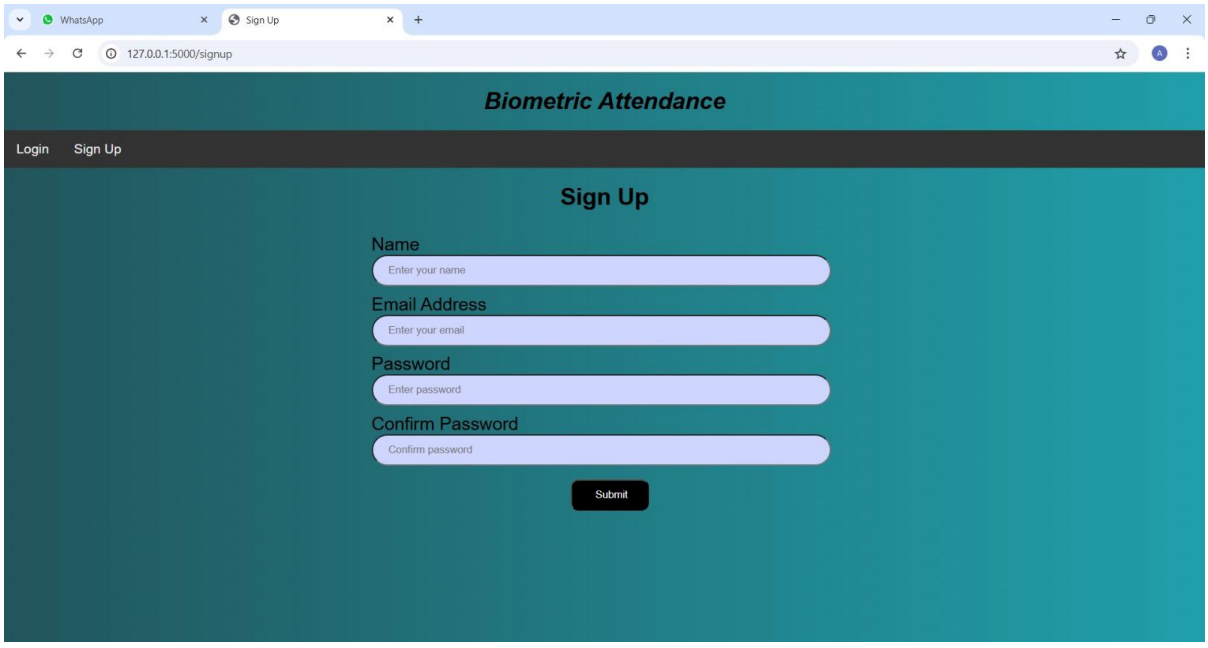
```
connection
  client.loop();

  // Check Wi-Fi connectivity and reconnect if needed
  if (WiFi.status() != WL_CONNECTED) {
    Serial.println("WiFi disconnected, attempting to reconnect...");
    connectAWS();
  }
}

// Enroll a fingerprint
bool enrollmentSuccess = false;

void enrollFingerprint() {
  Serial.print("Enrolling ID: "); Serial.println(currentId);

  if (currentId > maxId) {
    Serial.println("Maximum ID limit reached. Cannot enroll more fingerprints.");
    publishFingerprintId(currentId); // Publish only the fingerprint ID
    return;
  }

  int p = -1;
  while (p != FINGERPRINT_OK) {
    p = finger.getImage();
    switch (p) {
      case FINGERPRINT_OK:
        Serial.println("Image taken");
        break;
      case FINGERPRINT_NOFINGER:
        Serial.println("No finger detected");
        break;
      case FINGERPRINT_PACKETRECIEVEERR:
        Serial.println("Communication error");
        break;
      case FINGERPRINT_IMAGEFAIL:
        Serial.println("Imaging error");
        break;
      default:
        Serial.println("Unknown error");
        break;
    }
  }

  if (!convertToTemplate()) {
    Serial.println("Failed to create template. Enrollment aborted.");
    publishFingerprintId(currentId); // Publish only the fingerprint ID
    return;
  }

  p = finger.fingerSearch();
  if (p == FINGERPRINT_OK) {
                                        Serial.print("Fingerprint already enrolled with
```

```
ID: ");
        Serial.println(finger.fingerID);
        return;
    }

    Serial.println("Fingerprint not found. Proceeding with enrollment.");

    int storeResult = finger.storeModel(currentId);
    if (storeResult == FINGERPRINT_OK) {
        Serial.println("Fingerprint stored successfully!");
        enrollmentSuccess = true;
        currentId++;
    } else {
        Serial.println("Failed to store fingerprint.");
    }

    publishFingerprintId(currentId - 1); // Publish only the fingerprint ID
}

bool convertToTemplate() {
    int p = finger.image2Tz(1);
    if (p != FINGERPRINT_OK) {
        Serial.println("Failed to convert image to template");
        return false;
    }

    Serial.println("Remove finger and place it again");

    delay(2000);

    p = -1;
    while (p != FINGERPRINT_OK) {
        p = finger.getImage();
    }

    p = finger.image2Tz(2);
    if (p != FINGERPRINT_OK) {
        Serial.println("Failed to capture second image for template");
        return false;
    }

    p = finger.createModel();
    if (p != FINGERPRINT_OK) {
        Serial.println("Failed to create fingerprint model");
        return false;
    }

    return true;
}
```

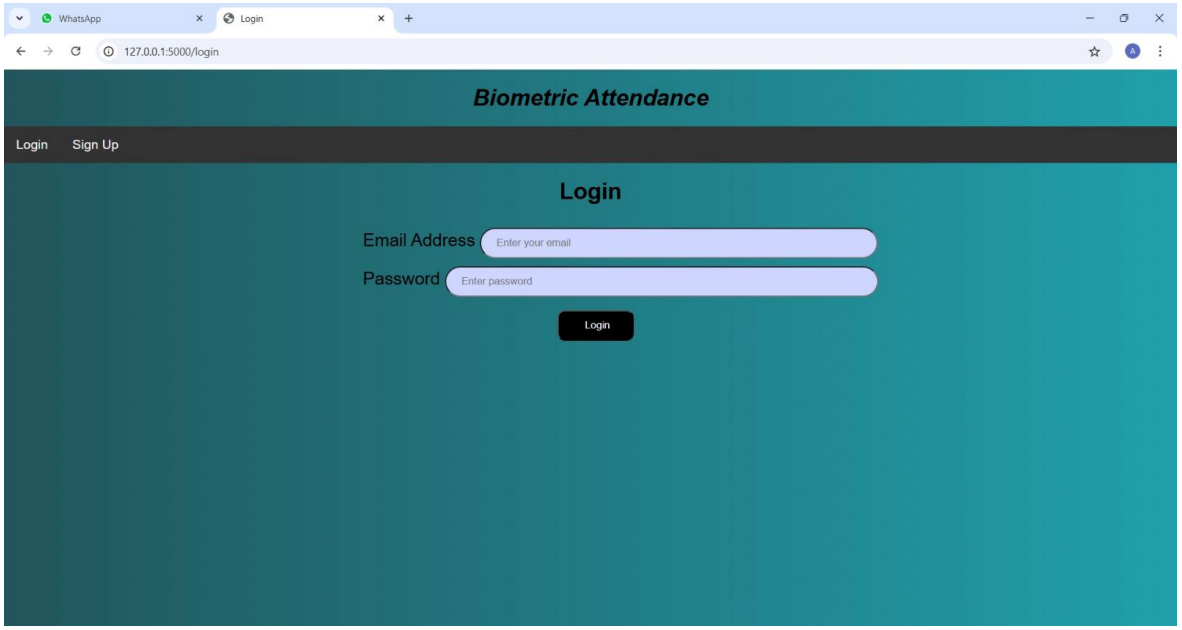


This screenshot illustrates the step of publishing fingerprint metadata, such as fingerprint id and template details, to AWS IoT Core.

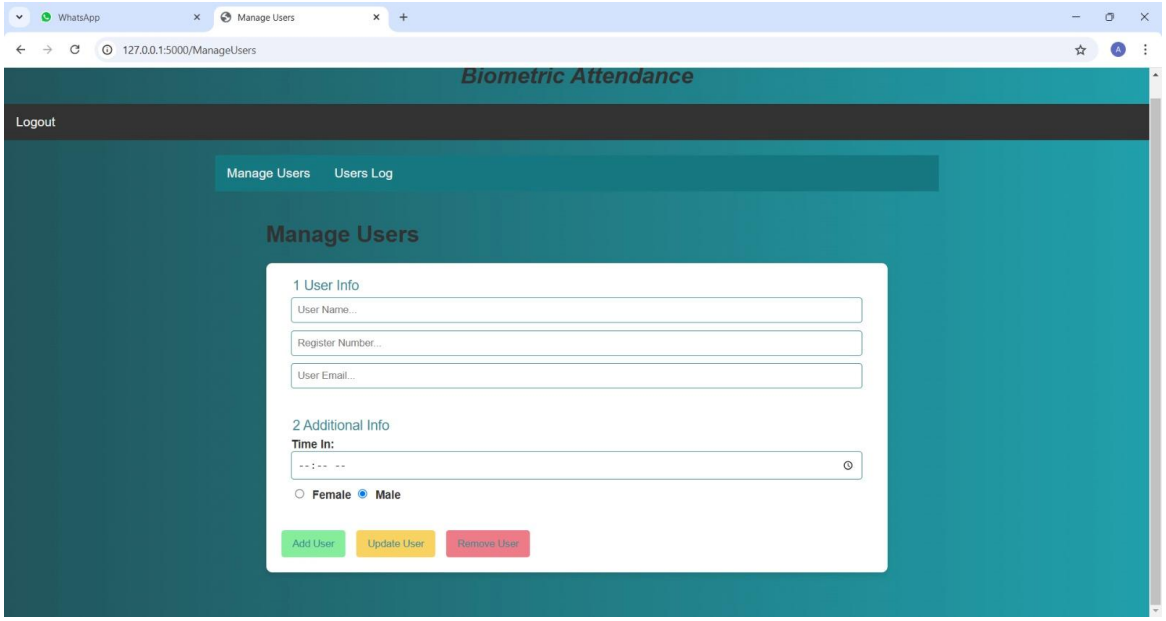
Web Interface Setup



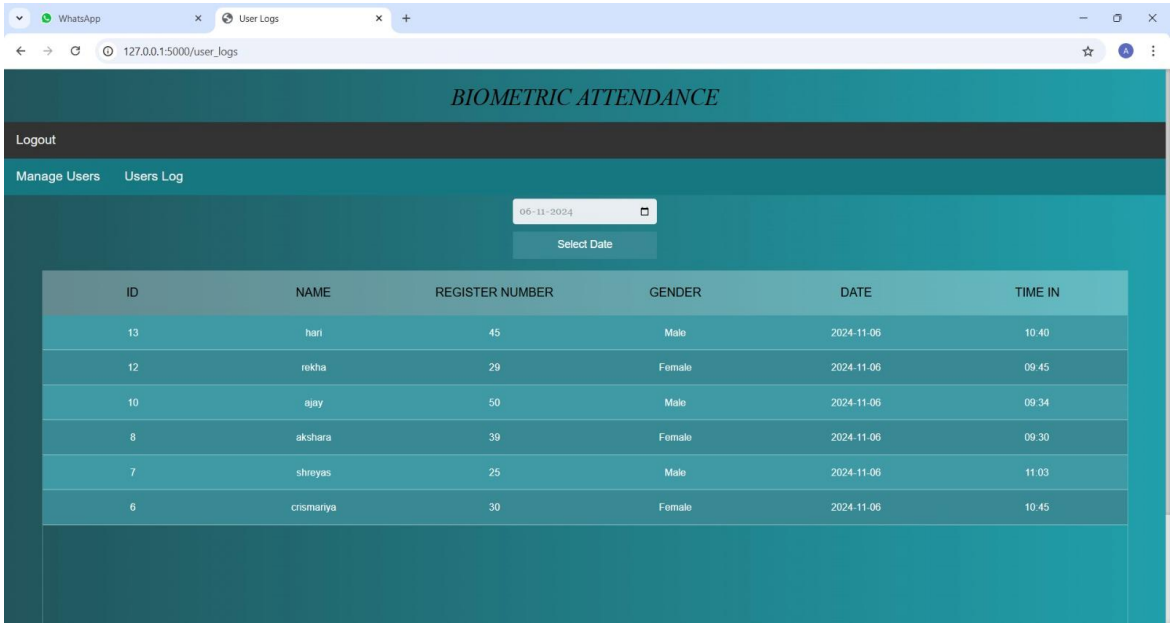
This screenshot shows the Sound Guard app's sign-up page for new users.



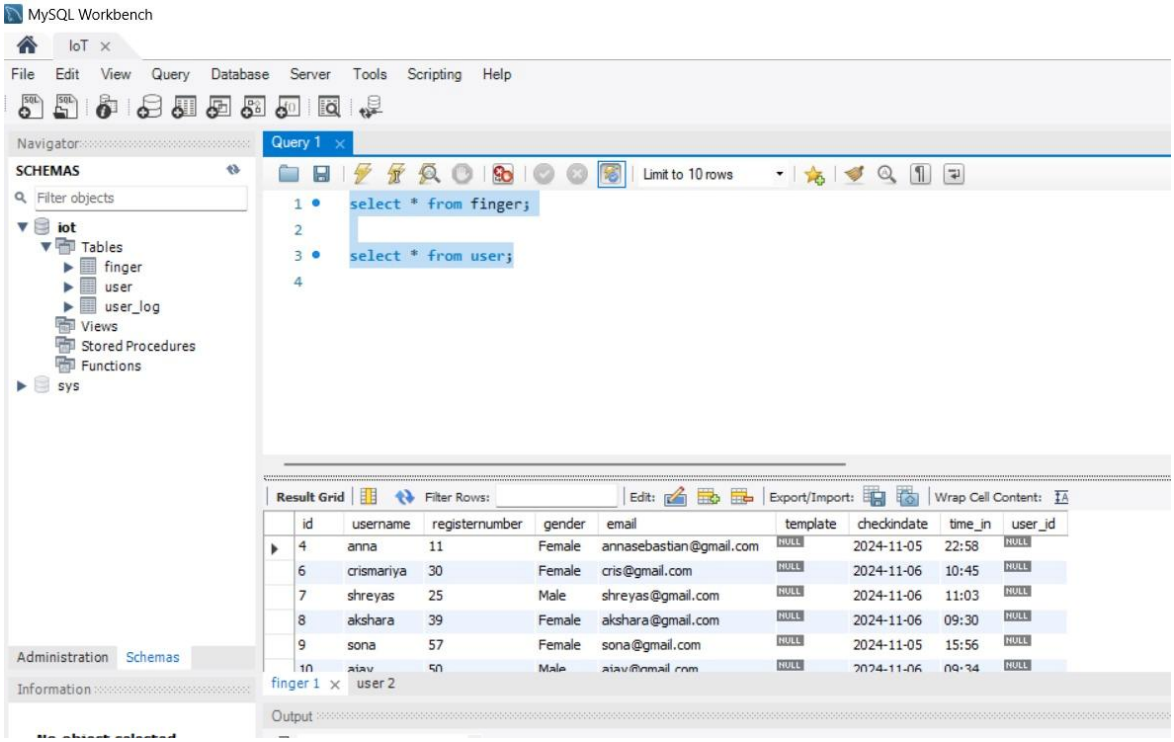
This displays the biometric attendance system's login page, where admin users can login by entering their credentials.



This is the manage users page, where admin can add users, update users and can remove users.



This is the users log page, where the admin can view the date wise attendance by choosing the day and month.



This shows the MySQL Workbench interface displaying the results of SQL queries.

6. CONCLUSION

The Biometric Attendance System project effectively integrates fingerprint recognition technology with IoT and cloud-based communication to provide a reliable and secure solution for attendance management. By utilizing the R307 fingerprint sensor and MQTT protocol, the system ensures accurate attendance tracking in real-time while maintaining data security through cloud storage. The web interface allows easy enrollment, verification, and management of user data, ensuring seamless integration into institutional environments. This project demonstrates the potential of biometric solutions in enhancing administrative processes, offering a scalable, secure, and efficient tool for managing attendance in educational institutions, workplaces, and beyond. Through the combination of *fingerprint authentication and cloud technologies, this system represents a significant advancement in automated attendance management.